Eye In-Painting with Exemplar Generative Adversarial Networks

Dolhansky, Brian, and Cristian Canton Ferrer, " *Proceedings of the IEEE conference on computer vision and pattern recognition",* 2018.

Adit Rastogi (2022A7PS1330H)

Srikanth Mujjiga (2024PHXP0507H)

Overview

Challenge:

- Traditional in-painting lacks **identity preservation**
  E.g: Generated eyes may not match the person's true eye shape/color).
- Humans are sensitive to facial inconsistencies ("uncanny valley").
- **One possible Solution**: ExGANs use **exemplar images/codes** to guide in-painting.

Key Idea:

- **Conditional GAN** variant leveraging exemplar data (reference images or perceptual codes).
- Two Variants
  - **Reference image based ExGANs:** Use a second image of the same person as guidance.
  - **Perceptual code based ExGANs:** compressed perceptual code (vector) from a reference image to capture identity traits.

Architecture

Generator:

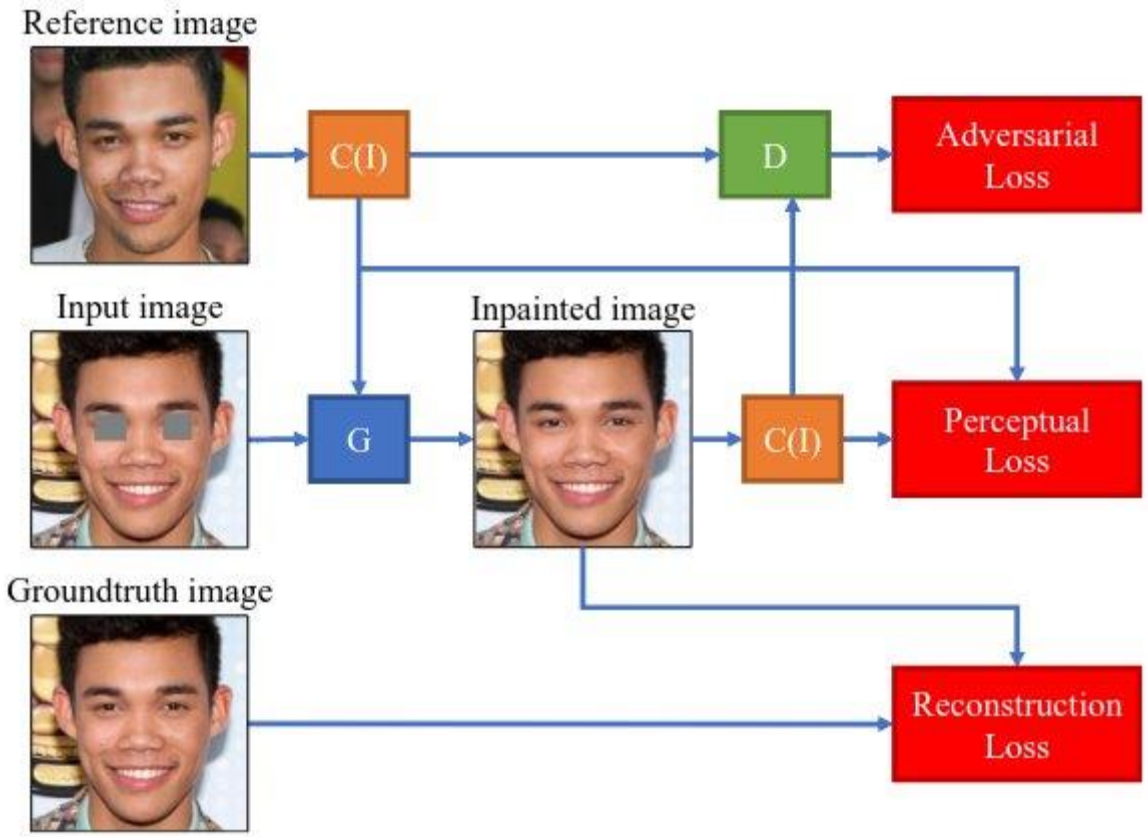
1. **Exemplar Encoder**: U-Net trained end to end as autoencoder
2. U-Net encoder layer later used for generating codes
3. Generated codes are concatenated with the convolved input inputs to generate the inpainted image

Discriminator: Dual-branch

1. Global: Evaluates full-face realism.
2. Local: Focuses on eye regions.

Loss Functions:

1. Adversarial Loss: Standard GAN min-max game.
2. Reconstruction Loss: $\|G(z_i, r_i) - x_i\|_1$
3. Perceptual Loss: Matches features in VGG space (optional).



$$\min_G \max_D V(D,G) = \mathbb{E}[\log D(x_i, r_i)] + \mathbb{E}[\log(1 - D(G(z_i, r_i)))] + \text{L1}$$

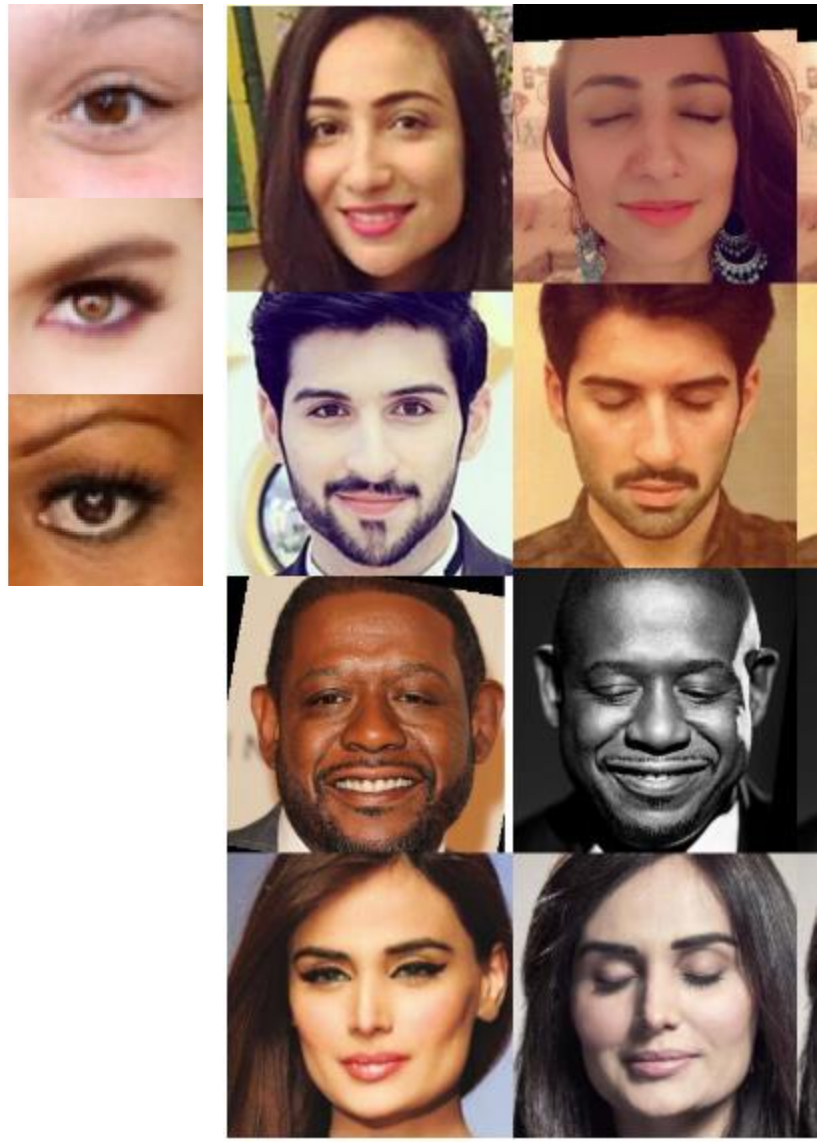
Dataset and Training

Dataset:

- Pairs of images (original + reference eyes) from celebrity photos.
- High resolution, varied lighting/poses.
- 2M image pairs

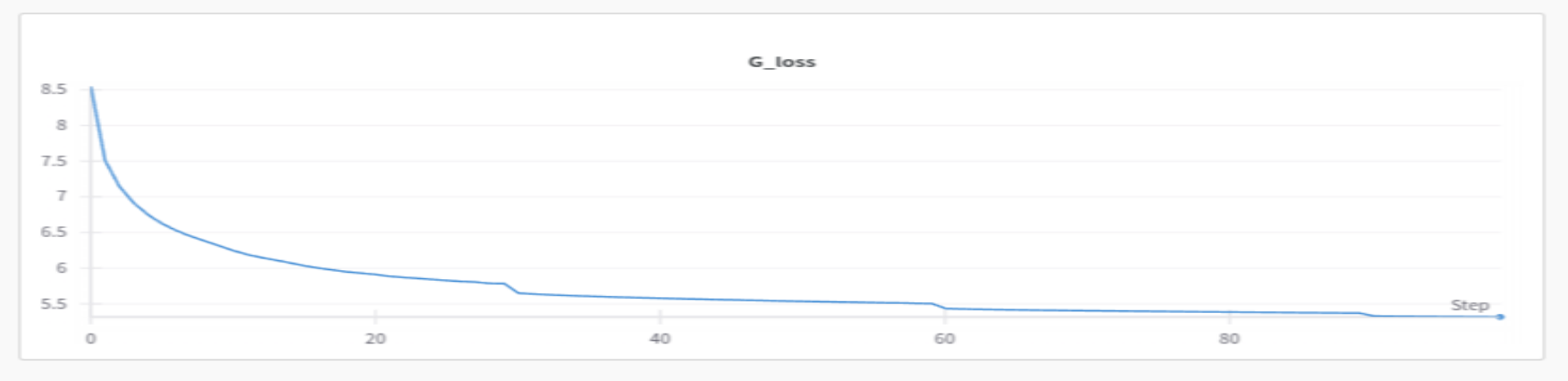
Training:

- Subsampled to 50K pairs
- Pretrained VGG for code generation
- Adam optimizer with lr=0.0002 and beta1=0.5, beta2=0.999, weight_decay=1e-4
- Scheduler: LR with step_size=30 and gamma=0.5
- Trained for 100 epochs. Took 2 days on 1 NVIDIA RTX A6000 (48GB)



Loss

Discriminator Loss



Generator Loss

Generator

```
class ExGAN(nn.Module):
    def __init__(self, z_dim=128, img_channels=3):
        super(ExGAN, self).__init__()
        self.generator = ExGANGenerator(img_channels=3, feature_channels=512)
        self.discriminator = ExGANDiscriminator(feature_channels=512)
        self.feature_extractor = FeatureExtractor()

        # Freeze feature extractor
        for param in self.feature_extractor.parameters():
            param.requires_grad = False

    def forward(self, z, exemplar_img):
        # Extract features from exemplar
        exemplar_features = self.feature_extractor(exemplar_img)

        # Generate image
        generated_img = self.generator(z, exemplar_features)
        generated_img_exemplar_features = self.feature_extractor(generated_img)
        return exemplar_features, generated_img, generated_img_exemplar_features
```

Discriminator

```
d_loss = losses.compute_adversarial_loss(exgan.discriminator,exemplar_features, fake_imgs_exemplar_features)
```

```
  def compute_adversarial_loss(
      self, discriminator, exemplar_features, fake_imgs_exemplar_features
  ):
      """Standard GAN adversarial loss"""
      batch_size = exemplar_features.size(0)

      # Real images loss
      pred_real = discriminator(exemplar_features)
      real_labels = torch.ones(batch_size, 1, device=self.device)
      loss_real = self.adversarial_loss(pred_real, real_labels)

      # Fake images loss
      pred_fake = discriminator(fake_imgs_exemplar_features.detach())
      fake_labels = torch.zeros(batch_size, 1, device=self.device)
      loss_fake = self.adversarial_loss(pred_fake, fake_labels)

      # Total discriminator loss
      d_loss = (loss_real + loss_fake) * 0.5

      return d_loss
```

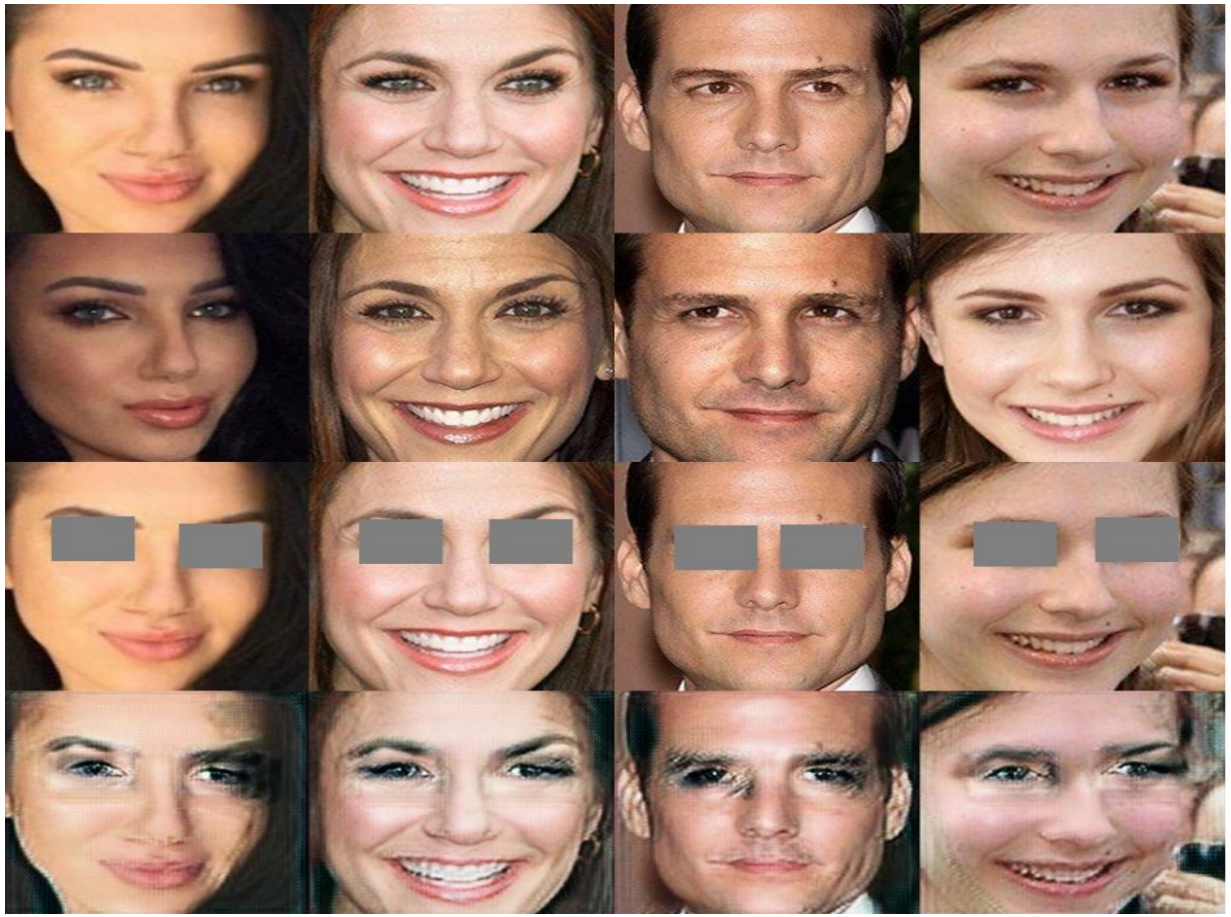
Training loop

```
for epoch in range(num_epochs):
    D_epoch_losses = []
    G_epoch_losses = []
    for real_imgs, masked_imgs, exemplar_imgs in dataloader:
        real_imgs = real_imgs.to(device)
        masked_imgs = masked_imgs.to(device)
        exemplar_imgs = exemplar_imgs.to(device)

        # Generate fake images
        exemplar_features, fake_imgs, fake_imgs_exemplar_features = exgan(
            masked_imgs, exemplar_imgs
        )
        # Update discriminator
        optimizer_d.zero_grad()
        d_loss = losses.compute_adversarial_loss(
            exgan.discriminator, exemplar_features, fake_imgs_exemplar_features
        )

        d_loss.backward()
        optimizer_d.step()

        # Update generator
        optimizer_g.zero_grad()
        g_losses = losses.compute_perceptual_loss(
            exemplar_features, fake_imgs_exemplar_features
        )
        r_losses = losses.compute_reconstruction_loss(fake_imgs, real_imgs)
        g_losses = g_losses + r_losses
        g_losses.backward()
        optimizer_g.step()
```

Results

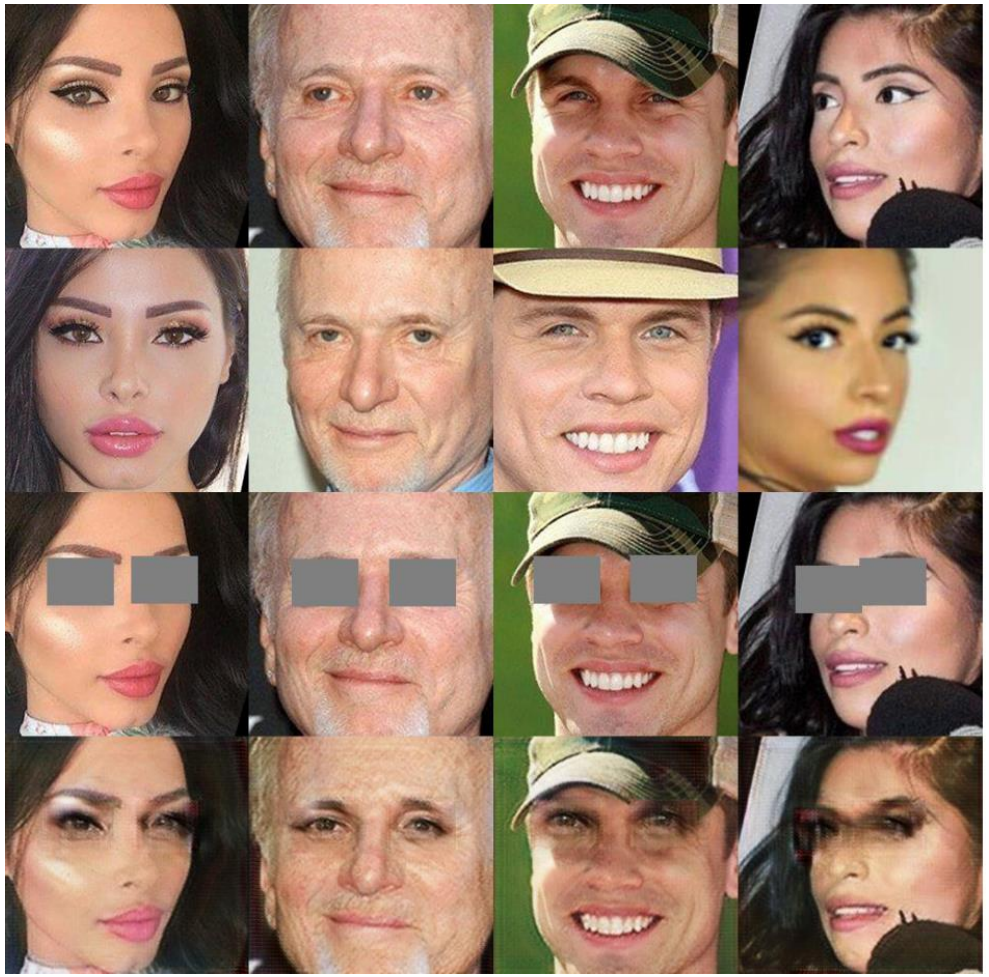
Step: 400

Step: 2000

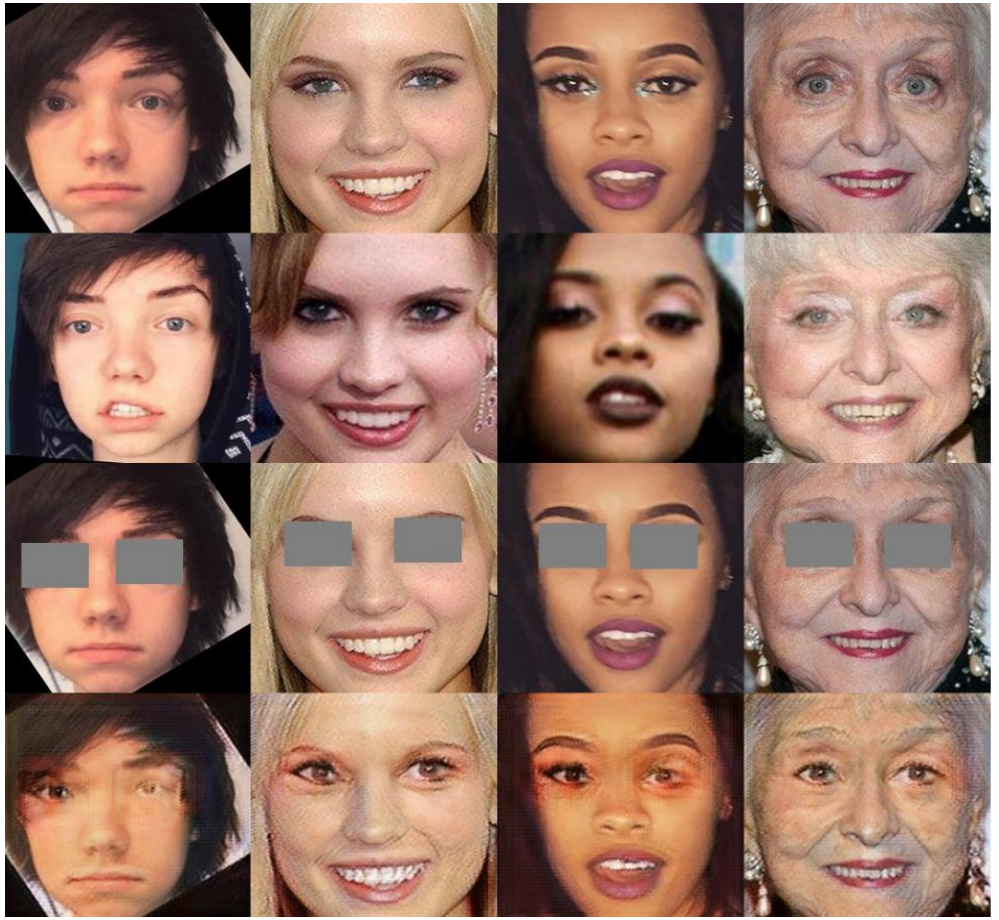


Results

Step: 4000



Step: 5600



Conclusion

Applications

- **Photo Editing**: Fix closed/blurred eyes in portraits.
- **VR/AR**: Generate realistic avatars from reference photos.
- **Medical Imaging**: In-paint missing regions in scans while preserving patient-specific features.

Challenges:

- Dataset: Square masks limit context
- Training: iris color mismatches
- Ethics: Potential misuse for deepfakes